Color and Texture Analysis Methods for Images

This document provides a summary of various methods used for analyzing color and texture in images, along with code snippets for each method.

1. Color Histogram

- Color Histograms represent the distribution of colors in an image.
- Useful for comparing the overall color distribution between images.

Code:

```python
import cv2
import numpy as np

def compute_color_histogram(image, bins=256):
    # Convert to HSV color space to capture hue, saturation, and value
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
    # Calculate histogram for H, S, V channels
    hist_hue = cv2.calcHist([hsv], [0], None, [bins], [0, 256])
    hist_saturation = cv2.calcHist([hsv], [1], None, [bins], [0, 256])
    hist_value = cv2.calcHist([hsv], [2], None, [bins], [0, 256])
    return hist_hue.flatten(), hist_saturation.flatten(), hist_value.flatten()
```

2. Color Moments

- Color Moments capture statistical properties (mean, variance, skewness) of colors in an image.
- Useful for measuring color consistency or comparing similar objects with different backgrounds.

Code:

```python
def compute_color_moments(image):
    # Convert image to HSV
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
    moments = cv2.moments(hsv)
    mean = np.mean(hsv, axis=(0, 1))
    variance = np.var(hsv, axis=(0, 1))
    skewness = np.mean((hsv - mean) ** 3, axis=(0, 1))  # Skewness approximation
    return mean, variance, skewness
```

3. Gabor Filters for Texture

- Gabor filters are used to extract texture features by analyzing frequency and orientation.
- Can capture edge-like structures, smooth textures, and more.

Code:

```python
import cv2
import numpy as np

def compute_gabor_texture(image):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
    # Define Gabor filter parameters
     kernels = [cv2.getGaborKernel((21, 21), 4.0, theta, 10.0, 0.5, 0, ktype=cv2.CV_32F) for theta in np.arange(0, np.pi, np.pi/4)]
    features = []
    for kernel in kernels:
        filtered = cv2.filter2D(gray, cv2.CV_8UC3, kernel)
        features.append(np.mean(filtered))
    return np.array(features)
```

4. Local Binary Patterns (LBP)

- LBP is a texture descriptor that captures local patterns by comparing pixel intensity to its neighbors.
- It is commonly used in texture classification.

Code:

```python
from skimage.feature import local_binary_pattern
import numpy as np

def compute_lbp_texture(image, radius=1, n_points=8):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    lbp = local_binary_pattern(gray, n_points, radius, method='uniform')
    # Calculate the histogram of LBP values
    hist, _ = np.histogram(lbp.ravel(), bins=np.arange(0, n_points + 3), range=(0, n_points + 2))
    return hist / hist.sum()  # Normalize the histogram
```

```

## 5. Haralick Texture Features

- Haralick features describe the texture by calculating statistical measures from the gray level co-occurrence matrix (GLCM).
- Commonly used for texture analysis in medical imaging.

Code:

```python
from skimage.feature import greycomatrix, greycoprops
import numpy as np

def compute_haralick_texture(image):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    # Compute the GLCM for different distances and angles
    glcm = greycomatrix(gray, [1], [0, np.pi/4, np.pi/2, 3*np.pi/4], symmetric=True, normed=True)
    # Extract Haralick features
    contrast = greycoprops(glcm, 'contrast')
    dissimilarity = greycoprops(glcm, 'dissimilarity')
    homogeneity = greycoprops(glcm, 'homogeneity')
    energy = greycoprops(glcm, 'energy')
    correlation = greycoprops(glcm, 'correlation')
        return np.hstack([contrast.mean(), dissimilarity.mean(), homogeneity.mean(), energy.mean(), correlation.mean()])
```
```

6. Entropy for Texture

- Entropy measures the randomness or complexity in the texture of an image.

- Higher entropy indicates more complex textures, lower entropy indicates simpler textures.

Code:

```python
from skimage.measure import shannon_entropy


def compute_entropy_texture(image):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    return shannon_entropy(gray)
```